

TECHNICAL DESIGN DOCUMENT

Three-Tier Counter Application on AWS

Infrastructure as Code with Terraform — CloudOps Assignment

Project: cloudops-assignment | Region: us-east-1 | Environment: dev

Field	Detail
Prepared by	Akash Chaudhari
Role	Platform / DevOps Engineer
Document type	Proposed Solution & Implementation Approach
Repository	bluepeak-cloudops-assignment
Provisioning method	Terraform

1. Proposed Solution

This document describes the proposed solution and implementation approach for deploying a resilient, scalable three-tier web application on AWS. The infrastructure is fully defined as code using Terraform, and follows AWS best practices for network isolation, least-privilege access, encryption at rest, and automatic recovery from instance-level failure.

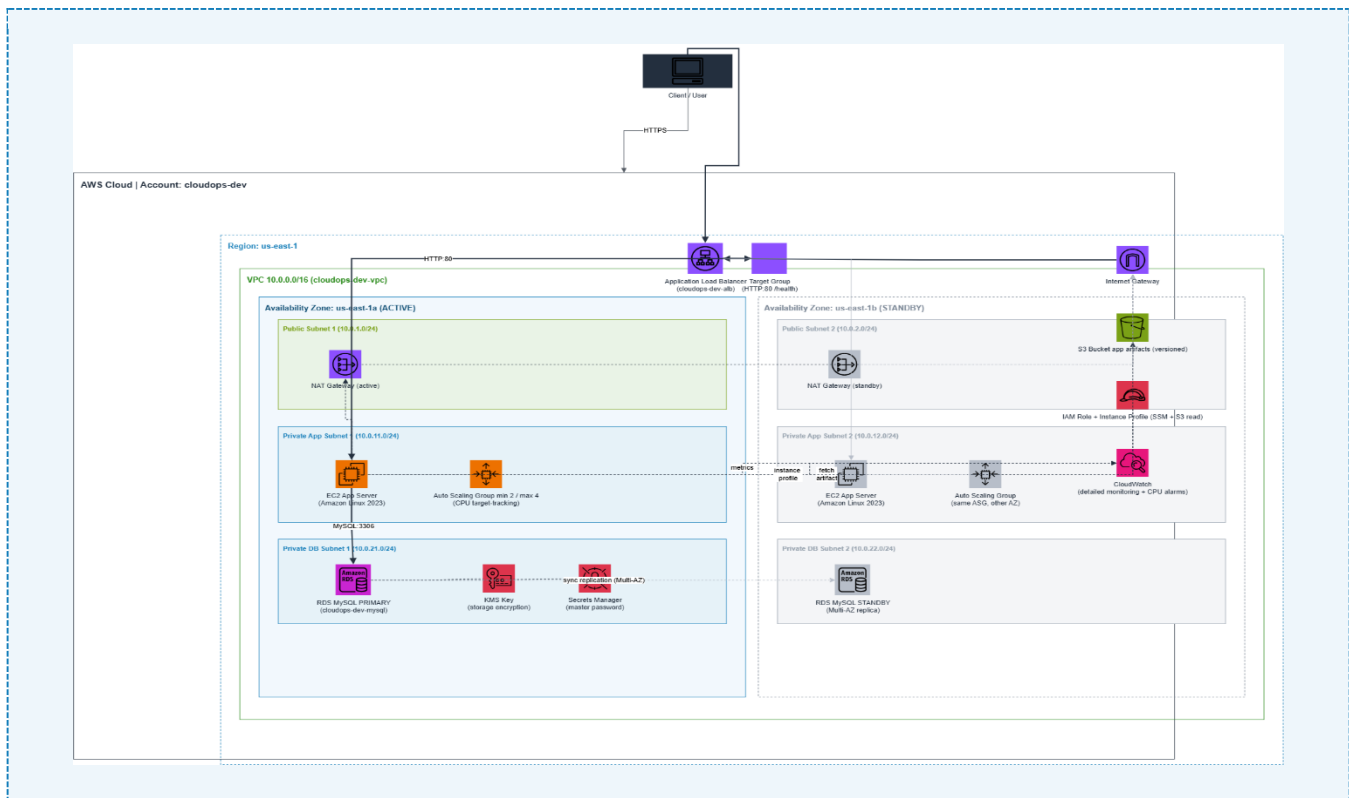
The workload is a lightweight containerised web application deployed behind an Application Load Balancer, running on Auto Scaling EC2 instances across two Availability Zones, backed by a Multi-AZ-capable Amazon RDS MySQL database. The goal of the assignment is to demonstrate a production-grade CloudOps pattern — modular IaC, tiered network segmentation, autoscaling, and secretless/keyless operational access rather than to build a feature-complete application.

2. Objectives

- Provision a secure, three-tier VPC (public / private-app / private-db) across two Availability Zones.
- Serve traffic through an internet-facing Application Load Balancer with health-checked targets.
- Run the application tier on an Auto Scaling Group that reacts to CPU utilisation, with zero long-lived SSH keys (Session Manager only).
- Persist application data in an encrypted, optionally Multi-AZ Amazon RDS MySQL instance.
- Keep every resource declarative and repeatable through modular Terraform, with remote state in S3.
- Package and version the application artifact through S3 rather than baking custom AMIs.

3. Proposed Architecture Overview

The architecture follows the classic three-tier model - presentation (ALB), application (EC2 Auto Scaling Group), and data (RDS) - contained inside a single VPC, split into public, private-application, and private-database subnets replicated across two Availability Zones for fault tolerance.



3.1 Request flow

The application is accessed directly using the **Application Load Balancer (ALB) DNS endpoint**, which serves as the only public entry point to the infrastructure.

When a user opens the ALB URL in a browser, the request reaches the **Application Load Balancer** over **HTTP (port 80)**. The ALB continuously performs health checks on the registered EC2 instances and forwards requests only to healthy targets in the Target Group.

The Target Group distributes incoming requests across EC2 instances running inside an **Auto Scaling Group**, which spans **two private application subnets** located in separate Availability Zones. This architecture ensures that application traffic is distributed evenly and that the application remains available even if one Availability Zone becomes unavailable.

Each EC2 instance is launched using an **EC2 Launch Template**. During instance initialization, the **User Data** script automatically performs the following actions:

- Installs Docker and required packages
- Downloads the latest **application.zip** from the Amazon S3 artifacts bucket
- Extracts the application files
- Builds the Docker image
- Starts the Counter application inside a Docker container

This automation ensures that every newly launched EC2 instance always runs the latest version of the application without requiring any manual deployment steps.

If the application needs to access persistent data, it securely communicates with the **Amazon RDS MySQL** database located in the **private database subnets**. Database access is allowed only from the EC2 Security Group on **port 3306**, ensuring that the database is never directly exposed to the internet.

Although the application servers are deployed in private subnets, they occasionally require outbound internet connectivity during startup to install operating system packages and download the application artifact from Amazon S3. This outbound communication is routed through the **NAT Gateway**, allowing internet access without exposing the EC2 instances to inbound public traffic.

Whenever application code is updated, Terraform uploads the new **application.zip** to the S3 artifacts bucket and updates the Launch Template by modifying the application version tag. This creates a new Launch Template version, which triggers an **Auto Scaling Group Instance Refresh**. The ASG gradually replaces existing instances with new ones, ensuring that the updated application is deployed with minimal downtime.

3.2 High availability model

The infrastructure is designed across two Availability Zones (us-east-1a and us-east-1b) to improve application availability and resiliency.

The Application Load Balancer is deployed across both public subnets and distributes incoming traffic to EC2 instances running in the private application subnets. The Auto Scaling Group spans both Availability Zones, ensuring that if an EC2 instance or an entire Availability Zone becomes unavailable, replacement instances can be launched automatically in the healthy Availability Zone.

For the database layer, Amazon RDS MySQL is deployed within private database subnets in both Availability Zones. Since this project is implemented within the AWS Free Tier and the sample Counter Application does not persist application data, Multi-AZ deployment is currently disabled to minimize infrastructure cost. However, the network and subnet architecture has been designed to support enabling Multi-AZ RDS in the future by simply updating the Terraform configuration (`multi_az = true`) without requiring any architectural changes.

4. AWS Services Used

Service	Purpose in this project
Amazon VPC	Network isolation - public, private-app, and private-db subnets across 2 AZs
Amazon EC2	Application tier compute (Amazon Linux 2023, Docker)
Amazon EC2 Auto Scaling	Maintains 2-4 healthy instances; scales on CPU target tracking
Elastic Load Balancing (ALB)	Internet-facing entry point, health checks, traffic distribution
Amazon RDS (MySQL)	Managed relational database, KMS-encrypted, Multi-AZ capable
AWS KMS	Customer-managed key for RDS storage encryption
AWS Secrets Manager	Auto-managed RDS master password (no password in state/config)

Service	Purpose in this project
Amazon S3	Versioned bucket for application build artifacts + Terraform remote state
AWS IAM	Least-privilege EC2 instance role (SSM + scoped S3 read only)
AWS Systems Manager (SSM)	Shell access to instances - no SSH keys, no open port 22
Amazon CloudWatch	Detailed EC2 monitoring and Auto Scaling metrics
Terraform (IaC)	Declarative, modular provisioning of every resource above

5. Repository & Infrastructure-as-Code Layout

The project is organised as a modular Terraform root with one module per functional layer, plus a standalone bootstrap stack for remote state:

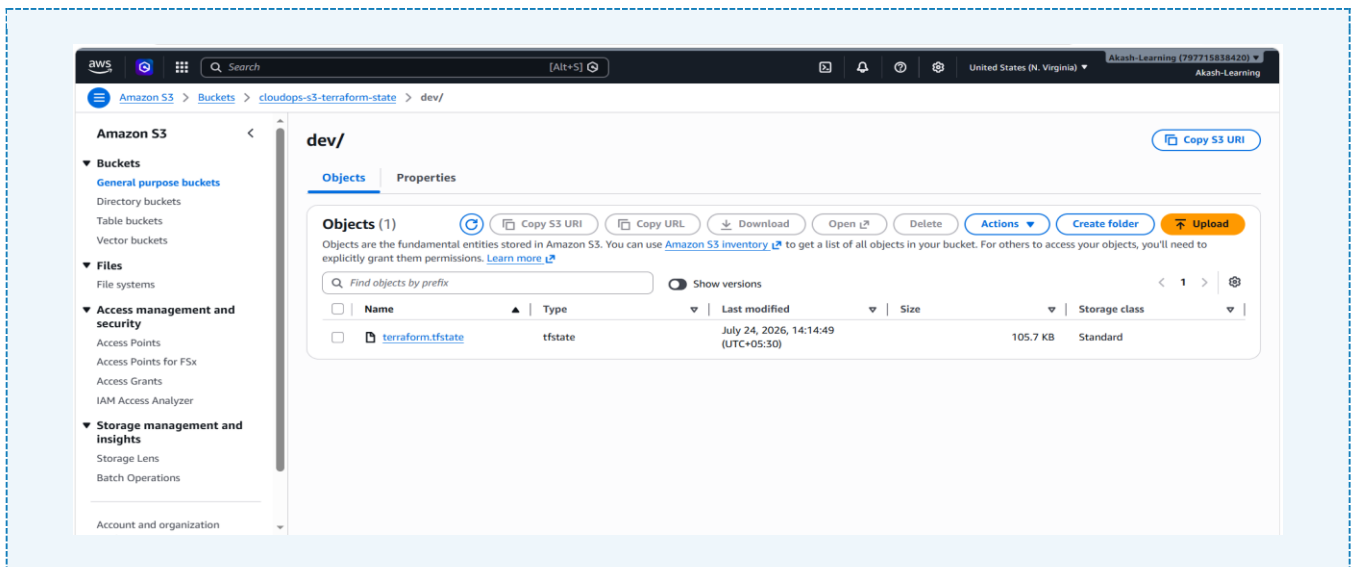
```
bluepeak-cloudops-assignment/
├── app/                # counter application + Dockerfile
├── s3-backend-bootstrap/ # One-time: creates the S3 state bucket
└── terraform/
    ├── main.tf         # Wires the modules together
    ├── variables.tf
    └── terraform.tfvars
        ├── backend.tf   # S3 remote state backend
        ├── provider.tf
        └── modules/
            ├── networking/ # VPC, subnets, IGW, NAT GW, route tables
            ├── security/   # ALB / EC2 / RDS security groups
            ├── alb/        # ALB, Target Group
            ├── ec2/        # AMI lookup, IAM role, Launch Template
            ├── autoscaling/ # Auto Scaling Group + scaling policy
            └── rds/        # DB subnet group, RDS instance, KMS key
```

6. Step-by-Step Implementation Approach

6.1 Remote State Bootstrap

Before the main stack runs once, the S3 bucket that stores Terraform's own state is created via the standalone s3-backend-bootstrap stack (versioning enabled, AES-256 server-side encryption, public access blocked).

```
cd s3-backend-bootstrap
terraform init
terraform apply
```



6.2 Networking (VPC module)

The networking module provisions a dedicated VPC (10.0.0.0/16) across two Availability Zones (us-east-1a and us-east-1b) to provide a highly available network foundation for the application.

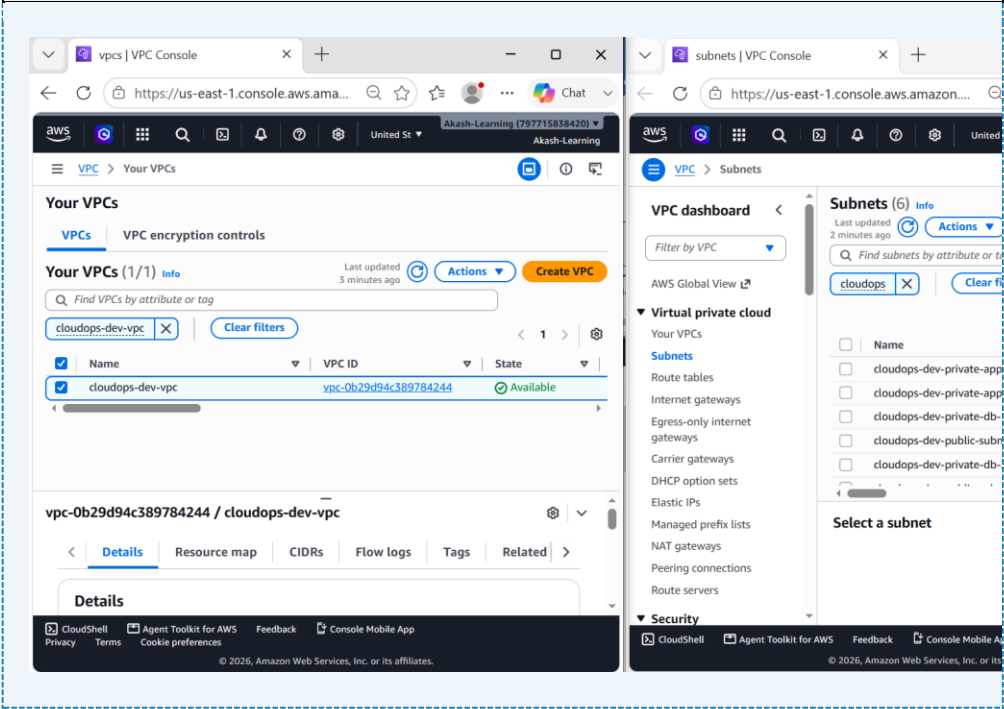
The infrastructure consists of:

- 1 VPC
- 1 Internet Gateway
- 1 NAT Gateway (deployed in a public subnet)
- 2 Public Subnets
- 2 Private Application Subnets
- 2 Private Database Subnets
- Separate Route Tables for Public, Application, and Database tiers

The Application Load Balancer is deployed in the public subnets to receive internet traffic. EC2 instances are launched by the Auto Scaling Group in the private application subnets, ensuring they are not directly accessible from the internet. The Amazon RDS database is deployed inside private database subnets for additional network isolation.

Subnet	AZ	CIDR	Tier
cloudops-dev-public-subnet-1	us-east-1a	10.0.1.0/24	Public
cloudops-dev-public-subnet-2	us-east-1b	10.0.2.0/24	Public

Subnet	AZ	CIDR	Tier
cloudops-dev-private-app-1	us-east-1a	10.0.11.0/24	Application
cloudops-dev-private-app-2	us-east-1b	10.0.12.0/24	Application
cloudops-dev-private-db-1	us-east-1a	10.0.21.0/24	Database
cloudops-dev-private-db-2	us-east-1b	10.0.22.0/24	Database

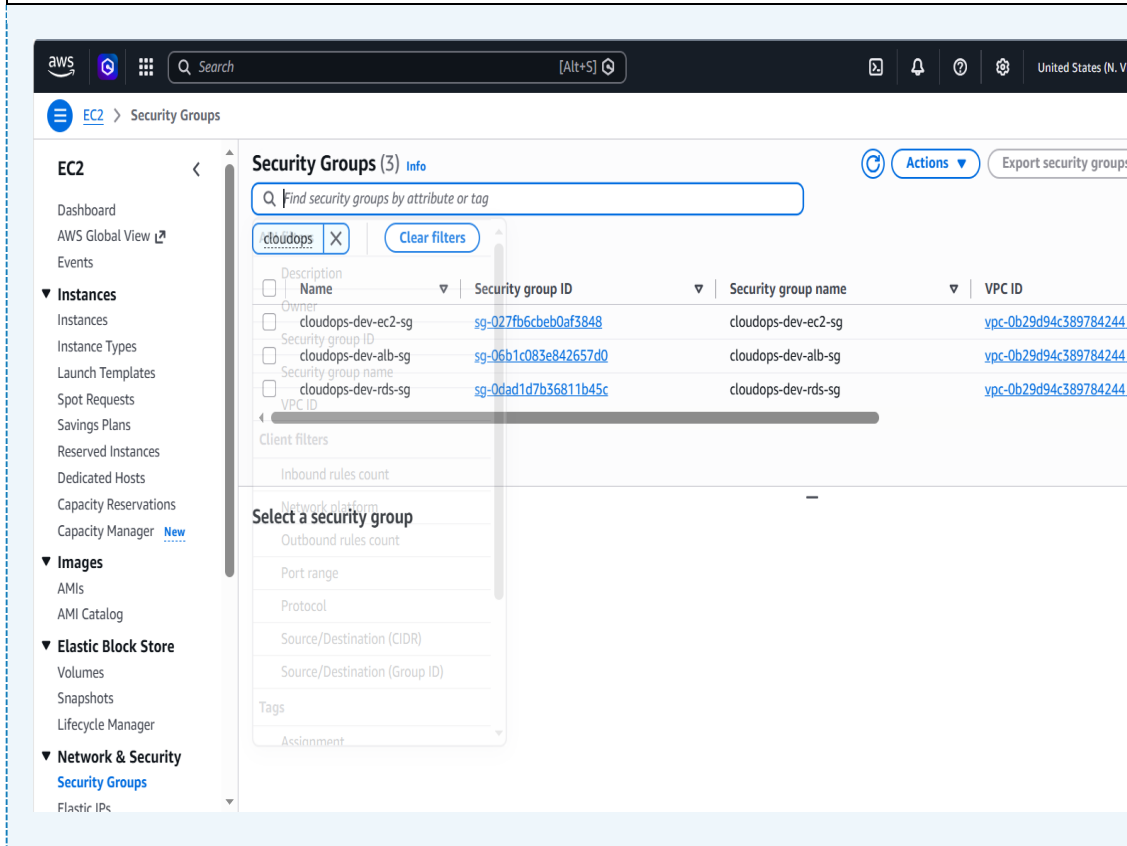


Subnet	AZ	CIDR	Tier																																							
Route table details VPC Manager https://us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#/route-tables Route tables (1/4) Find route tables by attribute or tag cloudops Clear filters <table><tr><th>Name</th><th>Route table ID</th><th>Explicit subnet associ...</th></tr><tr><td>cloudops-dev-private-db-rt</td><td>rtb-0e8f1af09d5818dc2</td><td>2 subnets</td></tr><tr><td>cloudops-dev-public-rt</td><td>rtb-09bee8bc48eb3fac2</td><td>2 subnets</td></tr></table> rtb-09bee8bc48eb3fac2 / cloudops-dev-public-rt Routes Subnet associations Edge associations Route propagation Routes (2) Filter routes <table><tr><th>Destination</th><th>Target</th><th>Status</th></tr><tr><td>0.0.0.0/0</td><td>igw-03d807302532a311b</td><td>Active</td></tr></table> Route table details VPC Manager https://us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#/route-tables Route tables (1/4) Find route tables by attribute or tag cloudops Clear filters <table><tr><th>Name</th><th>Route table ID</th><th>Explicit subnet associations</th></tr><tr><td>cloudops-dev-private-rt</td><td>rtb-0b7317590356bbb16</td><td>2 subnets</td></tr><tr><td>cloudops-dev-public-rt</td><td>rtb-0e1db1c7ebd01c7ebd0</td><td>2 subnets</td></tr></table> rtb-0b7317590356bbb16 / cloudops-dev-private-rt Details Routes Subnet associations Edge associations Route propagation Routes (2) Filter routes <table><tr><th>Destination</th><th>Target</th><th>Status</th></tr><tr><td>0.0.0.0/0</td><td>nat-097e53471827149e8</td><td>Active</td></tr></table> VPC dashboard Filter by VPC AWS Global View Virtual private cloud Your VPCs Subnets Route tables Internet gateways Egress-only internet gateways Carrier gateways DHCP option sets Elastic IPs Managed prefix lists NAT gateways Peering connections Route servers Security Network ACLs Security groups NAT gateways (1/1) Find NAT gateways by attribute or tag <table><tr><th>Name</th><th>NAT gateway ID</th><th>Connectivity...</th><th>State</th><th>State message</th><th>Availability</th></tr><tr><td>cloudops-dev-nat</td><td>nat-097e53471827149e8</td><td>Public</td><td>Available</td><td>-</td><td>Zonal</td></tr></table> nat-097e53471827149e8 / cloudops-dev-nat Details Secondary IPv4 addresses Monitoring Tags NAT gateway ID nat-097e53471827149e8 NAT gateway ARN arn:aws:ec2:us-east-1:797715838420:natgateway/nat-097e53471827149e8 VPC vpc-0b29d94c389784244 / cloudops-dev-vpc Connectivity type Public Primary public IPv4 address 107.23.246.102 Subnet subnet-0325ea43c8163f278 / cloudops-dev-public-subnet-1 State Available Primary private IPv4 address 10.0.1.52 Created Friday, July 24, 2026 at 14:08:24 GMT+5:30	Name	Route table ID	Explicit subnet associ...	cloudops-dev-private-db-rt	rtb-0e8f1af09d5818dc2	2 subnets	cloudops-dev-public-rt	rtb-09bee8bc48eb3fac2	2 subnets	Destination	Target	Status	0.0.0.0/0	igw-03d807302532a311b	Active	Name	Route table ID	Explicit subnet associations	cloudops-dev-private-rt	rtb-0b7317590356bbb16	2 subnets	cloudops-dev-public-rt	rtb-0e1db1c7ebd01c7ebd0	2 subnets	Destination	Target	Status	0.0.0.0/0	nat-097e53471827149e8	Active	Name	NAT gateway ID	Connectivity...	State	State message	Availability	cloudops-dev-nat	nat-097e53471827149e8	Public	Available	-	Zonal
Name	Route table ID	Explicit subnet associ...																																								
cloudops-dev-private-db-rt	rtb-0e8f1af09d5818dc2	2 subnets																																								
cloudops-dev-public-rt	rtb-09bee8bc48eb3fac2	2 subnets																																								
Destination	Target	Status																																								
0.0.0.0/0	igw-03d807302532a311b	Active																																								
Name	Route table ID	Explicit subnet associations																																								
cloudops-dev-private-rt	rtb-0b7317590356bbb16	2 subnets																																								
cloudops-dev-public-rt	rtb-0e1db1c7ebd01c7ebd0	2 subnets																																								
Destination	Target	Status																																								
0.0.0.0/0	nat-097e53471827149e8	Active																																								
Name	NAT gateway ID	Connectivity...	State	State message	Availability																																					
cloudops-dev-nat	nat-097e53471827149e8	Public	Available	-	Zonal																																					

6.3 Security Groups

Security is implemented using separate security groups for the Load Balancer, EC2 instances, and RDS database. The Application Load Balancer accepts HTTP traffic from the internet, while EC2 instances only allow inbound traffic from the ALB security group. Similarly, the RDS database only accepts connections from the EC2 security group. This layered approach ensures that the application and database tiers are isolated from direct public access and that communication is restricted to only the required components.

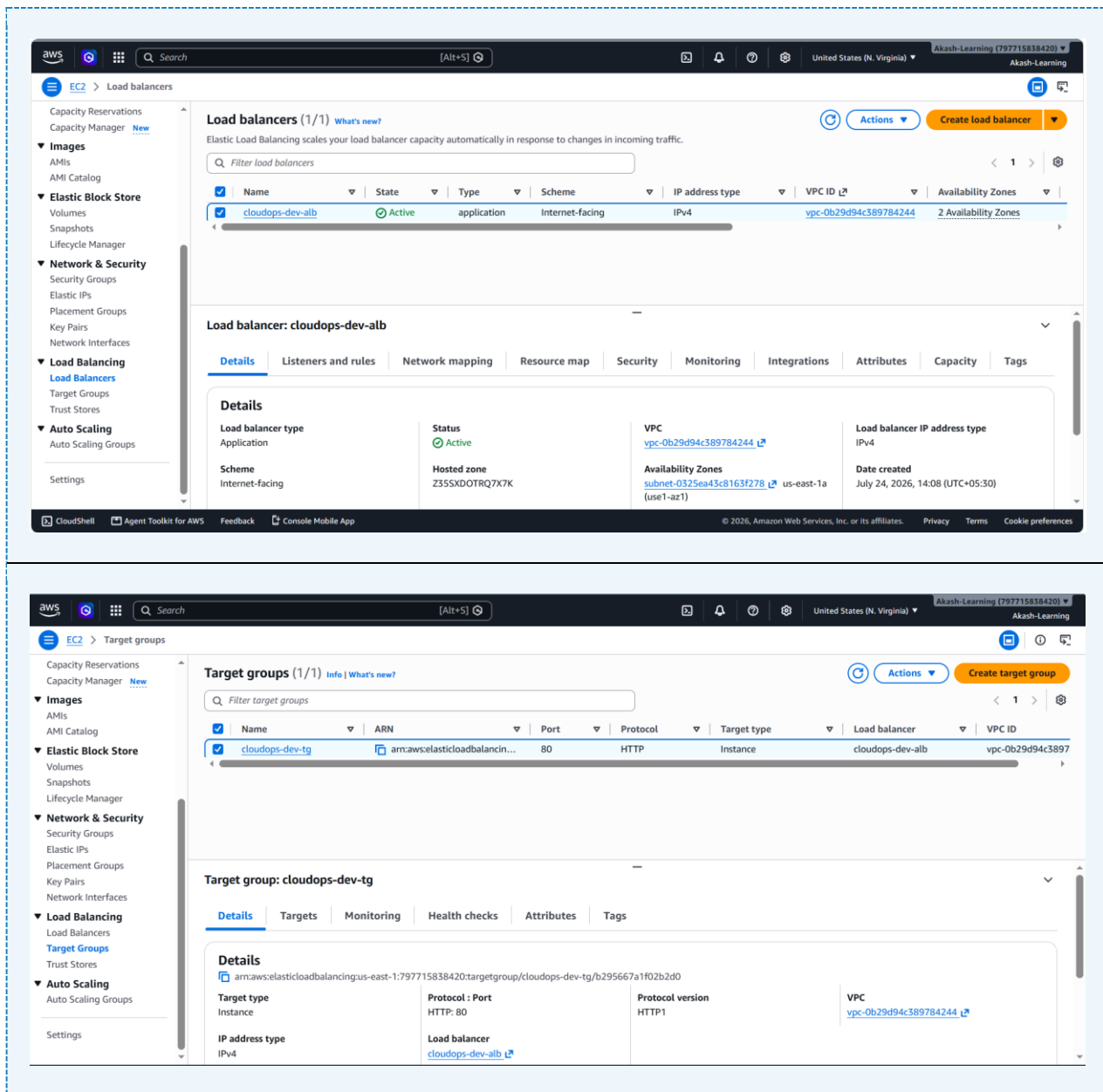
Security Group	Inbound rule	Source
cloudops-dev-alb-sg	TCP 80, TCP 443	0.0.0.0 /0
cloudops-dev-ec2-sg	TCP 80	cloudops-dev-alb-sg
cloudops-dev-rds-sg	TCP 3306 (MySQL)	cloudops-dev-ec2-sg



6.4 Application Load Balancer & Target Group

An internet-facing **Application Load Balancer (ALB)** is deployed across the two public subnets to receive incoming HTTP traffic from users. The ALB forwards requests to a single Target Group, where the EC2 instances launched by the Auto Scaling Group are automatically registered.

Health checks are configured on the Target Group to monitor the availability of the application running on the EC2 instances. If an instance becomes unhealthy, the ALB stops routing traffic to it, ensuring that requests are served only by healthy instances. This setup provides a reliable entry point for the application while distributing traffic across multiple EC2 instances.



6.5 Compute - Launch Template, IAM Role & User Data

The compute layer is built using an **Amazon EC2 Launch Template** and an **Auto Scaling Group** to ensure consistent instance provisioning and automated scaling. The Launch Template retrieves the **latest Amazon Linux 2023 AMI** during deployment and defines all the configuration required for every EC2 instance.

An IAM Role is attached to each instance through an Instance Profile. The role follows the principle of least privilege and provides only the permissions required by the application.

The IAM Role includes:

- **AmazonSSMManagedInstanceCore** managed policy, allowing secure administration of EC2 instances using AWS Systems Manager Session Manager without requiring SSH access.
- A custom inline policy that grants:

- `s3:GetObject` permission to download the application artifact (application.zip) from the Amazon S3 artifact bucket.
- `s3:ListBucket` permission on the artifact bucket to enable object access.

The Launch Template is configured with the following settings:

- **Amazon Linux 2023** as the base operating system.
- **30 GB encrypted gp3 EBS root volume** with automatic deletion when the instance is terminated.
- **Detailed CloudWatch monitoring** enabled for enhanced operational visibility.
- Required security groups and IAM Instance Profile attached during instance launch.

User Data Automation

A User Data script is executed automatically whenever a new EC2 instance is launched by the Auto Scaling Group.

The script performs the complete application deployment without requiring manual intervention by:

1. Updating the operating system packages.
2. Installing Docker, AWS CLI, and required utilities.
3. Starting and enabling the Docker service.
4. Downloading the latest application.zip artifact from the Amazon S3 artifact bucket using the EC2 IAM Role.
5. Extracting the application files.
6. Building the Docker image locally.
7. Running the Counter Application container on **port 80**.

This automated bootstrap process ensures that every newly launched instance is configured consistently and always deploys the latest application version stored in Amazon S3. Combined with the Auto Scaling Group and Launch Template, this provides a repeatable and fully automated deployment workflow.

Dockerfile

```
FROM nginx:1.27-alpine

# Remove default nginx page
RUN rm -rf /usr/share/nginx/html/*

# Copy application files
COPY index.html /usr/share/nginx/html/
COPY style.css /usr/share/nginx/html/
COPY script.js /usr/share/nginx/html/

EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

UserData file

```
#!/bin/bash
```

```
set -euxo pipefail
```

```
APP_VERSION="${app_version}"
```

```
echo "Deploying version ${app_version}"
```

```
exec >>(tee /var/log/user-data.log)
```

```
exec 2>&1
```

```
dnf update -y
```

```
dnf install -y docker unzip awscli
```

```
systemctl enable docker
```

```
systemctl start docker
```

```
mkdir -p /opt/${app_name}
```

```
cd /opt/${app_name}
```

```
echo "Downloading application..."
```

```
aws s3 cp s3://${artifact_bucket}/${artifact_object_key} application.zip
```

```
echo "Unzipping..."
```

```
unzip -o application.zip
```

```
echo "Building Docker image..."
```

```
docker build -t ${app_name}:latest .
```

```
docker rm -f ${app_name} || true
```

```
echo "Starting container..."
```

```
docker run -d \
```

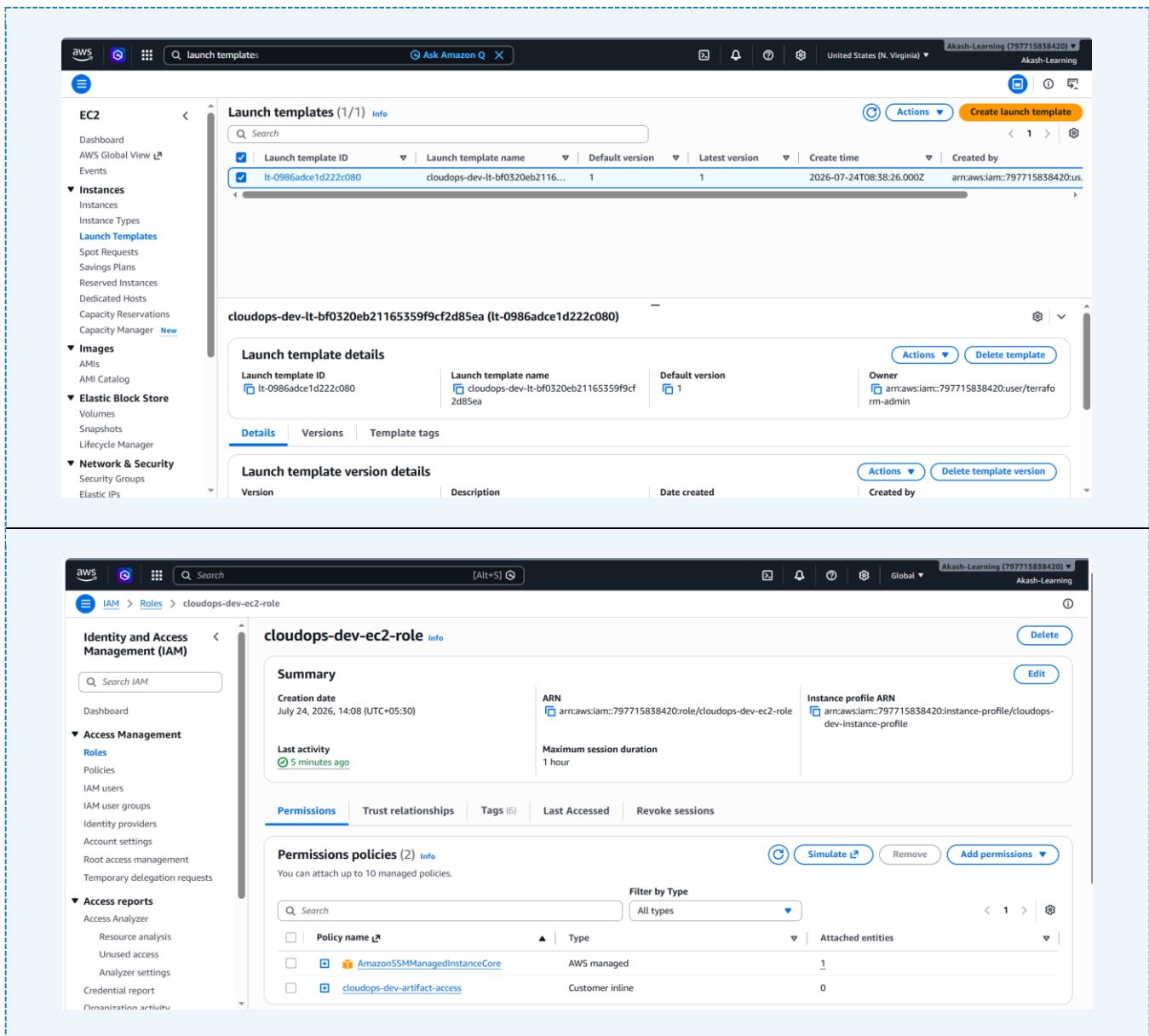
```
--name ${app_name} \
```

```
--restart unless-stopped \
```

```
-p 80:80 \
```

```
${app_name}:latest
```

```
echo "Deployment completed."
```



6.6 Auto Scaling Group

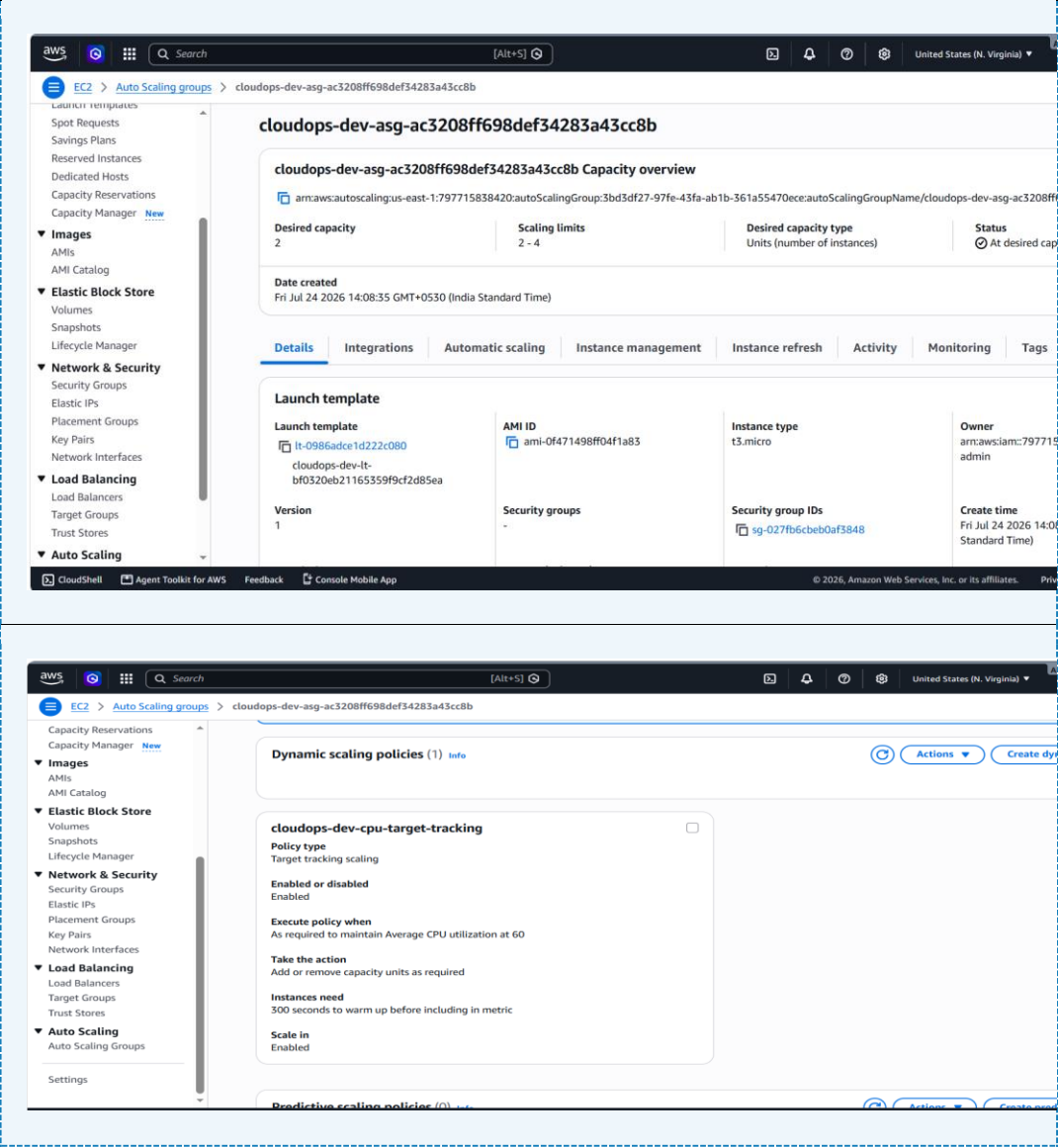
The application is deployed using an **Amazon EC2 Auto Scaling Group** across two private application subnets to provide high availability. The Auto Scaling Group is associated with the **Application Load Balancer Target Group**, allowing incoming requests to be distributed only to healthy EC2 instances.

To improve reliability, the Auto Scaling Group uses **ELB health checks** instead of only EC2 instance status checks. This ensures that if the application running inside an EC2 instance becomes unhealthy, the instance is automatically replaced even if the EC2 instance itself is still running.

The infrastructure also supports **rolling instance refreshes** whenever the Launch Template is updated. During a new deployment, the Auto Scaling Group gradually replaces existing instances with new ones while maintaining at least **50% healthy instances**, minimizing application downtime.

This configuration provides automatic scaling based on application load while ensuring high availability and seamless deployments with minimal service interruption.

Setting	Value
Min / Desired / Max	2 / 2 / 4
Health check type	ELB
Scaling policy	Target tracking — ASGAverageCPUUtilization
Deployment strategy	Rolling instance refresh on launch-template change (50% min healthy)

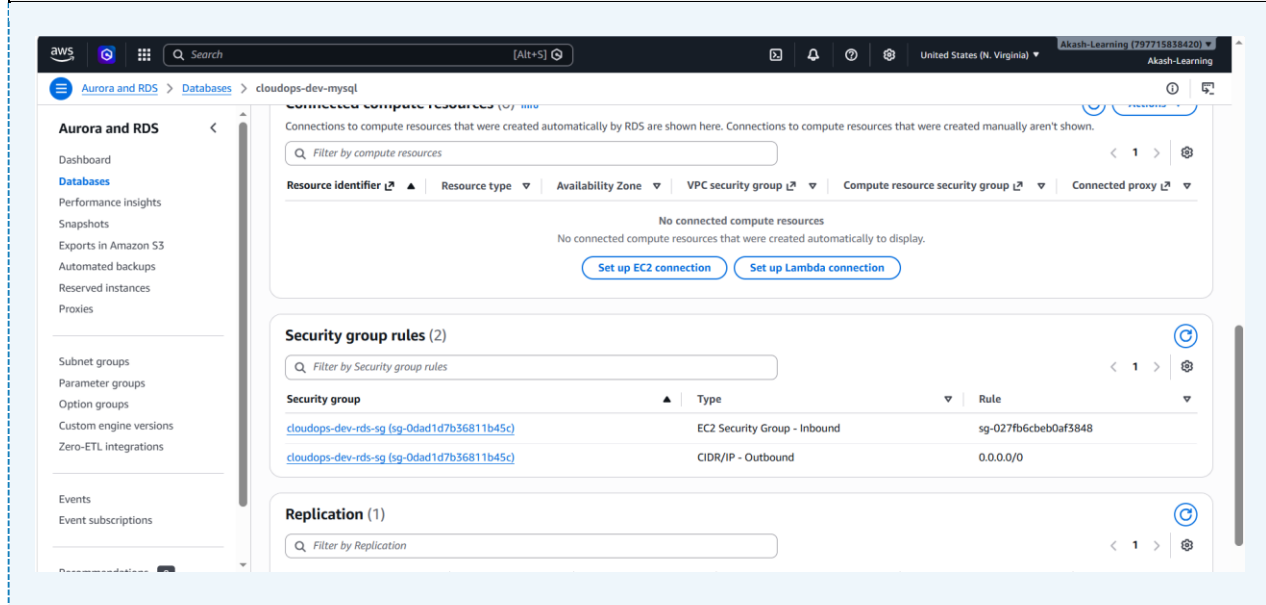


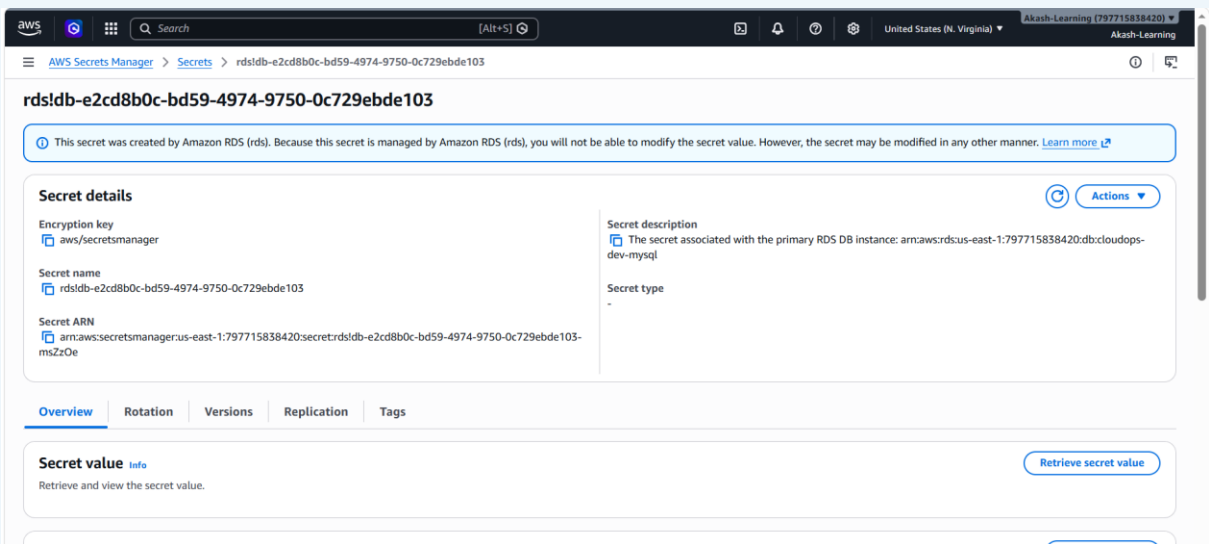
6.7 Database - Amazon RDS (MySQL)

The database layer is implemented using **Amazon RDS for MySQL 8.0**, deployed within dedicated **private database subnets** across two Availability Zones using an RDS DB Subnet Group. This ensures the database is not directly accessible from the internet and can only be reached by the application EC2 instances through the configured security groups.

To enhance security, the RDS instance uses **encryption at rest** with a dedicated **AWS KMS customer-managed key (CMK)**, with automatic key rotation enabled. Instead of hardcoding database credentials in Terraform code or configuration files, the master password is automatically generated and securely stored in **AWS Secrets Manager**. The secret ARN is exposed as a Terraform output, allowing applications or future services to retrieve the credentials securely without exposing sensitive information in the infrastructure code.

Setting	Value
Engine	MySQL 8.0
Instance class	db.t3.micro
Storage	20 GB, gp3, KMS-encrypted
Multi-AZ	Configurable (off in dev, on in prod)
Public access	Disabled
Credential storage	AWS Secrets Manager (manage_master_password)



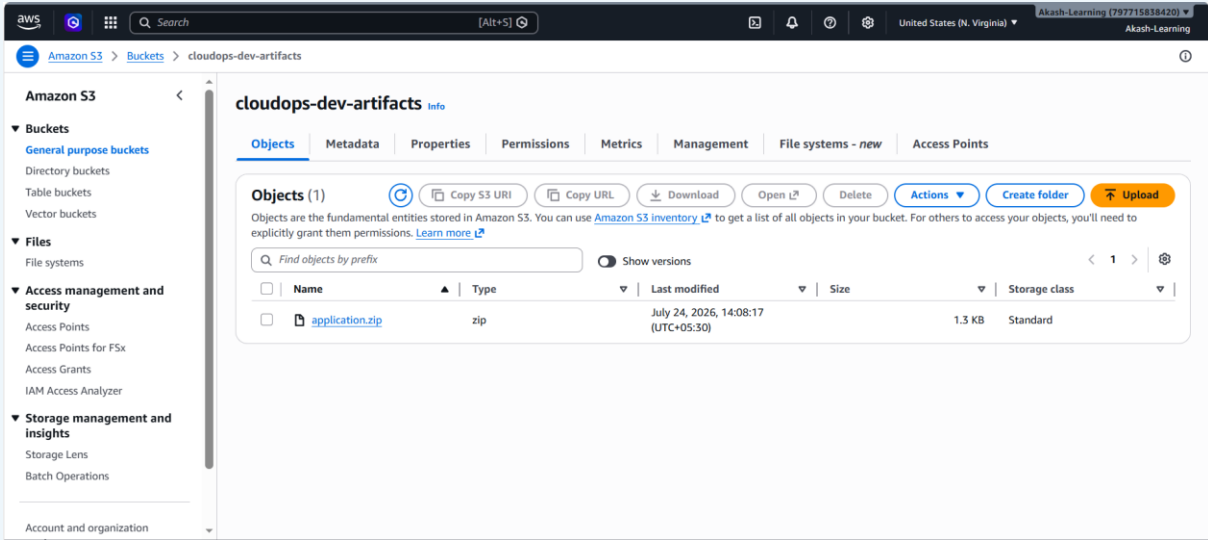
Setting	Value
	

6.8 Artifact Storage — Amazon S3

The Counter Application source code is packaged into **application.zip** using Terraform's `archive_file` data source and uploaded to a private, versioned Amazon S3 bucket.

During EC2 instance initialization, the **User Data** script downloads the latest application artifact from the S3 bucket, extracts it, builds the Docker image, and starts the container automatically.

Whenever the application code is updated, running **terraform apply** uploads a new version of **application.zip** to S3. This triggers the creation of a new Launch Template version, allowing the Auto Scaling Group to launch EC2 instances with the updated application. This approach eliminates the need for manually copying files to servers or creating custom AMIs for every application update.

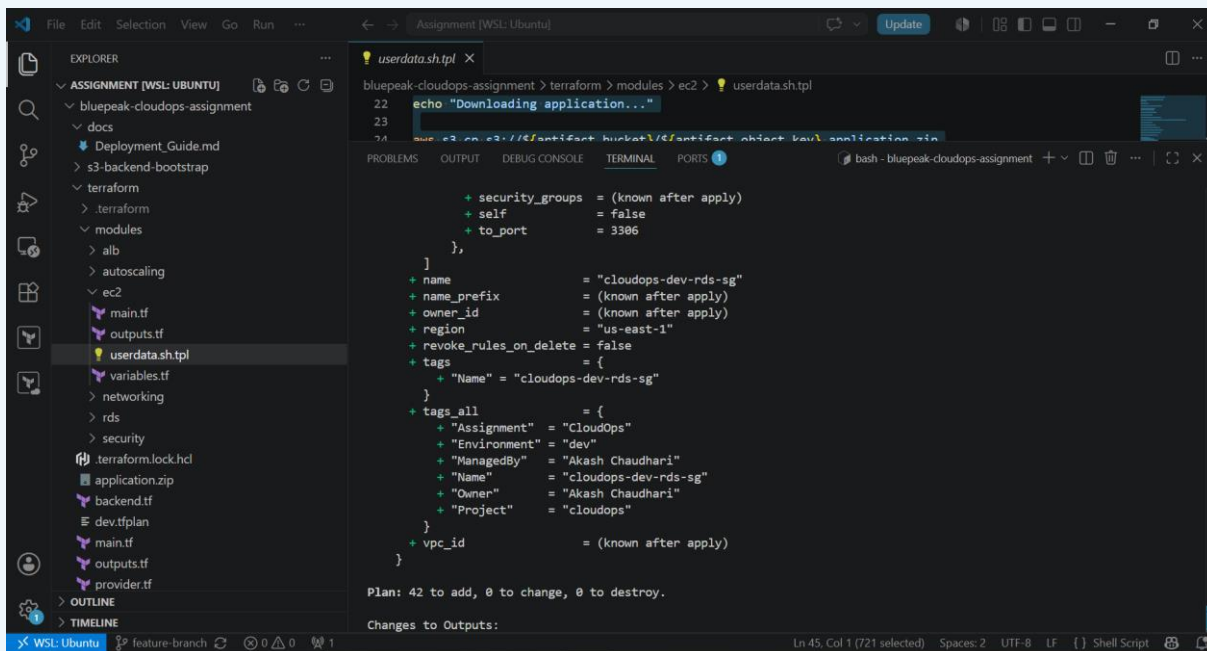
	
--	--

6.9 Deploying with Terraform

After the remote S3 backend has been created and configured, the complete AWS infrastructure can be provisioned from the `terraform/` directory using the standard Terraform workflow. This deployment creates all the required resources, including the networking components, security groups, Application Load Balancer, Auto Scaling Group, Launch Template, Amazon RDS, IAM roles, and the S3 artifact bucket. During the deployment, the `application.zip` file is uploaded to Amazon S3, and the Launch Template is configured with a User Data script that automatically downloads the application, builds the Docker image, and starts the container whenever a new EC2 instance is launched.

Run the following commands from the `terraform/` directory:

```
cd terraform
terraform init
terraform plan
terraform apply
```



```

bluepeak-cloudops-assignment > terraform > modules > ec2 > userdata.sh.tpl
22 echo "Downloading application..."
23
24 aws ec2 run-instances --profile bluepeak --region us-east-1 --ami ami-0c55b1e9 --instance-type t3.micro --key-name bluepeak --security-groups sg-01234567 --user-data file://./userdata.sh.tpl --output s3://bluepeak-cloudops-assignment/terraform/outputs/outputs.tf

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - bluepeak-cloudops-assignment

module.rds.aws_db_instance.main: Still creating... [05m00s elapsed]
module.rds.aws_db_instance.main: Still creating... [05m10s elapsed]
module.rds.aws_db_instance.main: Still creating... [05m20s elapsed]
module.rds.aws_db_instance.main: Still creating... [05m30s elapsed]
module.rds.aws_db_instance.main: Still creating... [05m40s elapsed]
module.rds.aws_db_instance.main: Still creating... [05m50s elapsed]
module.rds.aws_db_instance.main: Still creating... [06m00s elapsed]
module.rds.aws_db_instance.main: Still creating... [06m10s elapsed]
module.rds.aws_db_instance.main: Creation complete after 6m16s [id=db-SCWTCTUNW0IROOOQUVIPHAWLLE]

Warning: 'launch_template' always triggers an instance refresh and can be removed

with module.autoscaling.aws_autoscaling_group.main,
on modules/autoscaling/main.tf line 23, in resource "aws_autoscaling_group" "main":
23:   triggers = [
24:     "launch_template"
25:   ]

Releasing state lock. This may take a few moments...

Apply complete! Resources: 42 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "cloudops-dev-alb-2131588138.us-east-1.elb.amazonaws.com"
autoscaling_group_name = "cloudops-dev-asg-ac3208ff698def34283a43cc8b"

```

7. Testing & Validation

7.1 Functional test

After the infrastructure was successfully deployed, the **Application Load Balancer (ALB)** DNS name generated by Terraform was used to access the application.

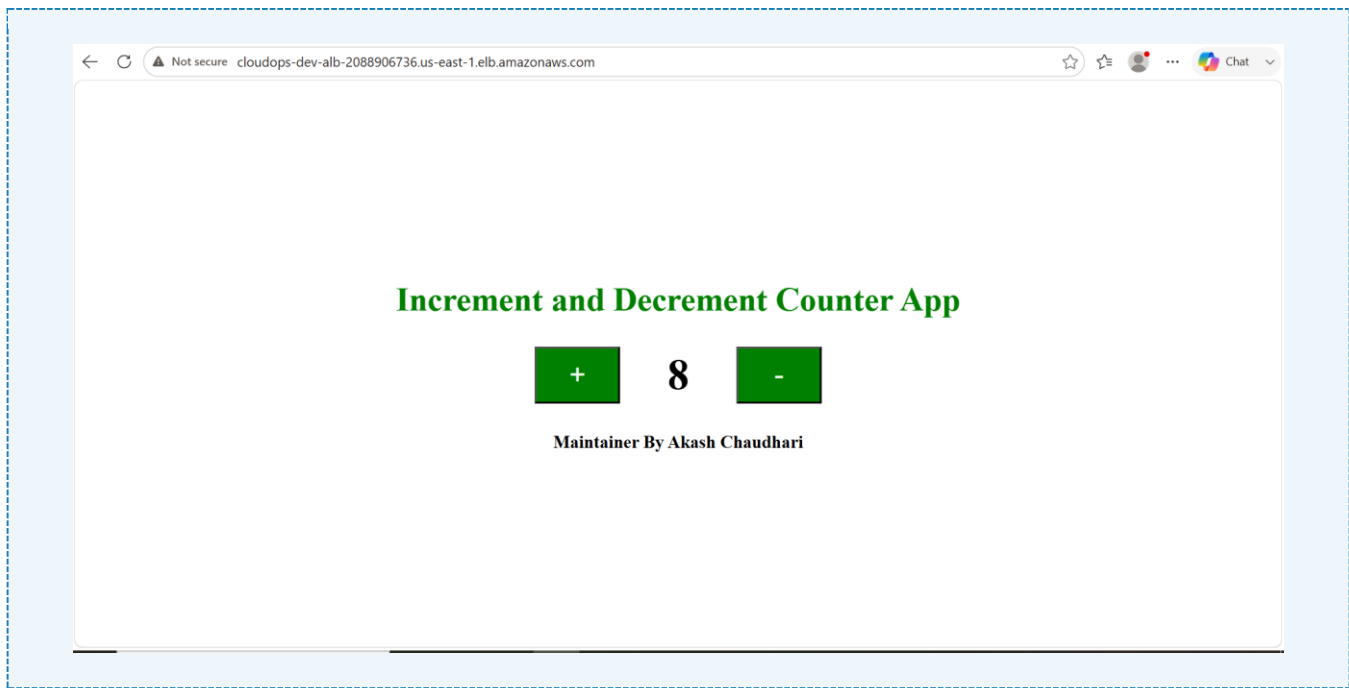
Open the ALB DNS URL in a web browser:

`http://<alb_dns_name>`

The Counter Application should load successfully, displaying the **Increment** and **Decrement** buttons.

Perform the following validation:

- Verify that the application loads without any errors.
- Click the **Increment** button and confirm that the counter value increases.
- Click the **Decrement** button and confirm that the counter value decreases.
- Refresh the page to ensure the application continues to respond correctly.



This verifies that:

- The Application Load Balancer is correctly routing incoming requests.
- The Target Group is forwarding traffic only to healthy EC2 instances.
- The EC2 instances successfully downloaded the latest application package from the S3 artifact bucket during startup.
- The Docker container was built and started successfully using the User Data script.
- The Counter Application is accessible and functioning correctly through the public ALB endpoint.

8. Security Considerations

- No inbound SSH anywhere - all shell access is via SSM Session Manager, logged centrally.
- Three-tier security-group chaining - each tier only accepts traffic from the tier directly above it.
- RDS is never publicly accessible and sits in dedicated private subnets with no route to the Internet Gateway.
- Storage encrypted at rest: RDS via a customer-managed KMS key, EBS root volumes encrypted by default.
- Database credentials are never stored in Terraform state/tfvars - Secrets Manager owns the master password lifecycle.
- IAM least privilege - the EC2 role can read exactly one S3 object, nothing broader.

9. Proposed Future Enhancements

The current stack is intentionally single-region to keep the assignment's cost and scope manageable. If this were promoted toward a production, disaster-recovery-ready design, the following would be the natural next additions - noted here as proposed, not yet implemented:

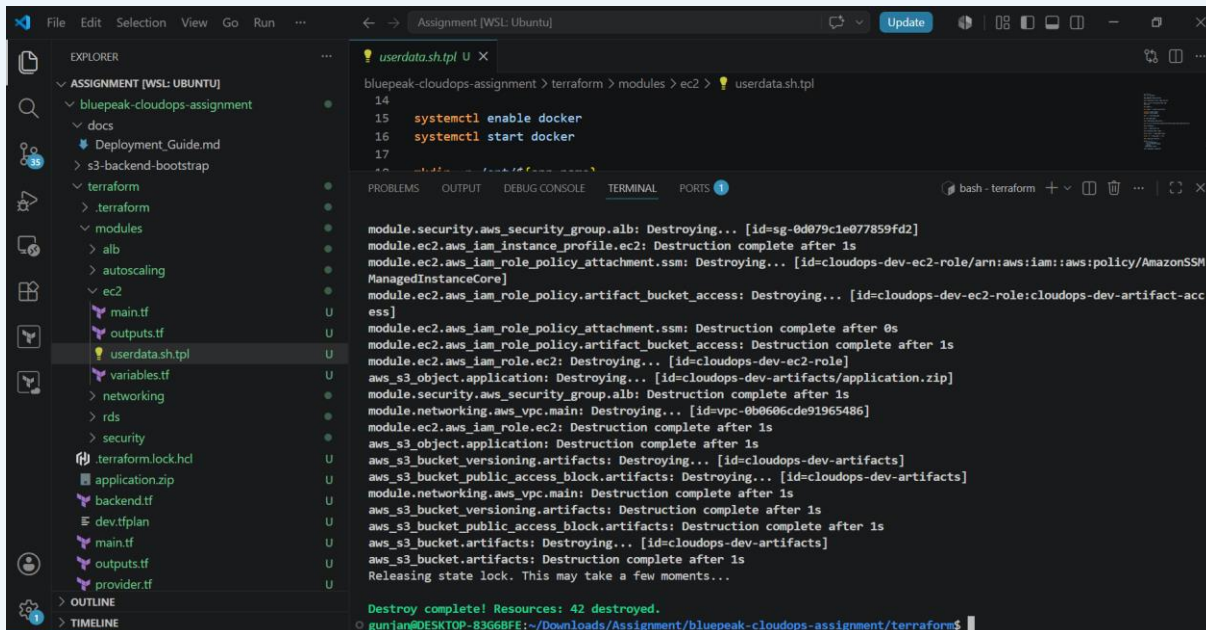
- Amazon Route 53 with health-check-based failover routing to a secondary region.
- Amazon CloudFront in front of the ALB for edge caching and a second layer of regional failover.
- AWS WAF attached to CloudFront/ALB for bot and exploit protection.
- Cross-region read replica for RDS, promotable during a regional failover.
- AWS Backup with cross-region copy for EC2/EBS and automated recovery point retention.
- HTTPS listener on the ALB via AWS Certificate Manager once a custom domain is attached.
- Enabling multi_az = true on RDS for automatic same-region failover in production.

10. Resource Cleanup

Once the project is no longer required, all AWS resources can be removed using Terraform.

```
cd terraform/
terraform destroy

# only if the remote-state bucket itself should also be removed:
cd ..
cd s3-backend-bootstrap/
terraform destroy
```



11. Conclusion

This solution provides a secure and scalable three-tier architecture on AWS using Terraform. The infrastructure is deployed across two Availability Zones with a dedicated VPC, public and private subnets, an internet-facing Application Load Balancer, an Auto Scaling Group running Dockerized EC2 instances, and an Amazon RDS MySQL database hosted in private subnets. Application artifacts are stored in Amazon S3, and EC2 instances automatically download and deploy the latest version during startup using a User Data script. Sensitive database credentials are securely managed through AWS Secrets Manager, while Terraform state is stored remotely in Amazon S3 with native S3 state locking (`use_lockfile = true`). The entire environment can be recreated consistently using Terraform, providing a reliable, secure, and cost-effective foundation for development. As future requirements grow, the architecture can be extended with services such as Route 53, CloudFront, AWS WAF, HTTPS using ACM, and multi-region disaster recovery without major changes to the existing infrastructure.